CECS 440 – Lab 7

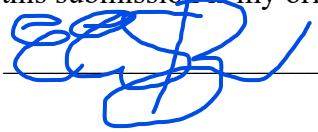
Write Back

Due date: 11/4/2025

Name: Eric Santana

ID: 015107467

I certify that this submission is my original work.



Goal

The goal of this lab was to implement the Write Back stage of a MIPS processor. Implement control signals RegWriteM, and MemtoRegW to manage data flow to the register file. Connect and verify data paths for the ALUOutW, ReadData, and WriteRegM, and lastly simulate the design.

Steps

1. Opened Vivado and created a new project for lab 7.
2. Create the mux, and WB_Stage design modules.
3. Wrote and completed the Verilog code for each module according to the lab instructions.
4. Created a testbench to simulate the functionality of the design.
5. Ran the simulation and observed waveform and console output.

Results

The simulation showed that the Write Back stage correctly selected the correct data source based on the MemtoRegW signal. When MemtoRegW = 0, the ALU output was written back to the register file. When MemtoRegW = 1, the data read from memory was written instead. The RegWriteW signal was asserted correctly to enable register updates.

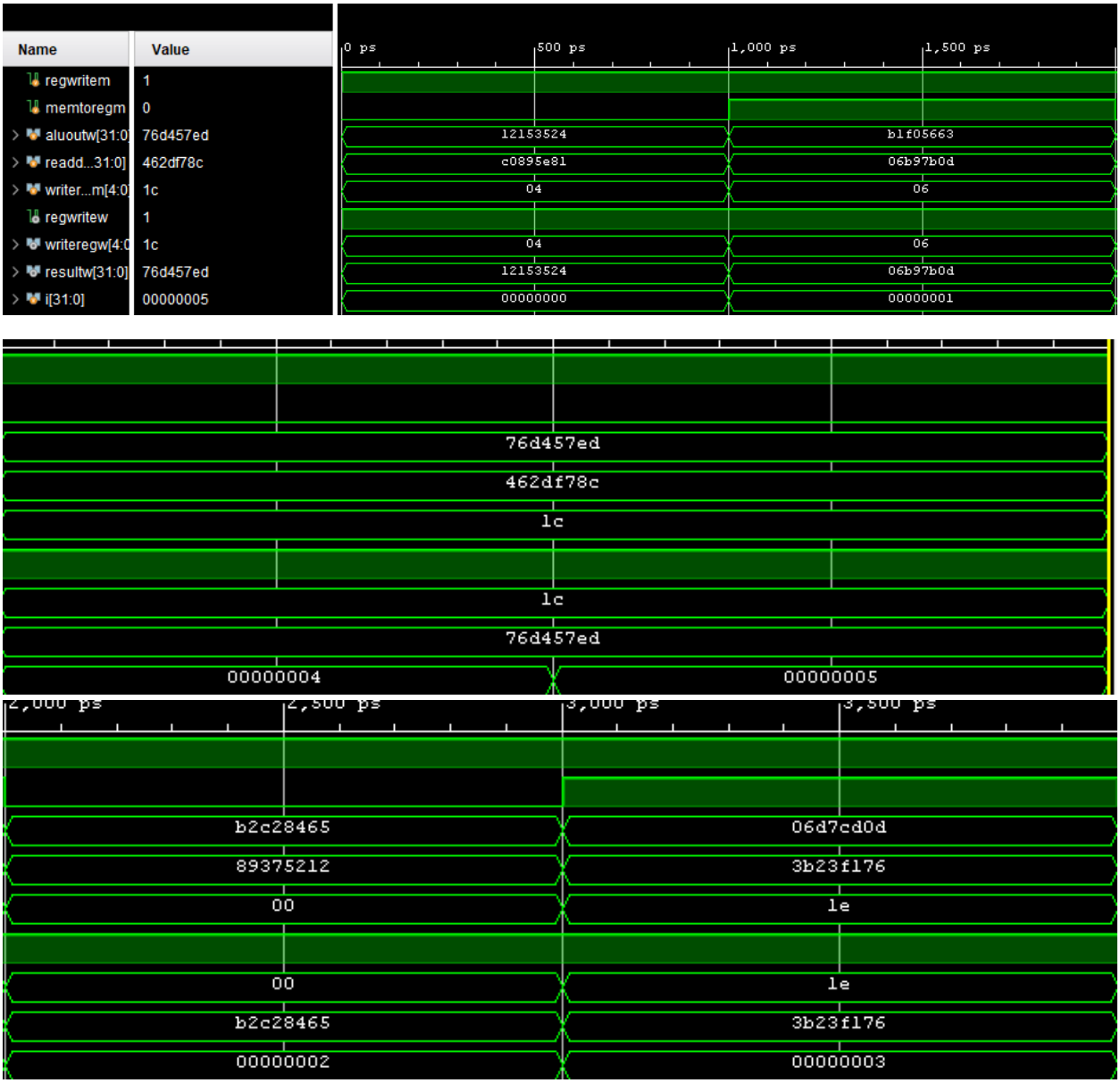
An issue I had was syntax errors on the design modules, but after I fixed those problems, I was able to successfully simulate the lab.

Conclusion

This lab helped strengthen my understanding of the MIPS datapath in regards to the Write Back stage. It helped me understand how control signals and a bus work together to move data from different sources in a processor.

Appendix (1 out of 3)

Waveforms



Appendix (2 out 3)

Console Output

```
Test Case           0
aluoutw = 12153524  readdata = c0895e81
memtoregm = 0   resultw = 12153524
regwritem = 1   regwritew = 1
writeregm = 04  writeregw = 04

Test Case           1
aluoutw = b1f05663  readdata = 06b97b0d
memtoregm = 1   resultw = 06b97b0d
regwritem = 1   regwritew = 1
writeregm = 06  writeregw = 06

Test Case           2
aluoutw = b2c28465  readdata = 89375212
memtoregm = 0   resultw = b2c28465
regwritem = 1   regwritew = 1
writeregm = 00  writeregw = 00

Test Case           3
aluoutw = 06d7cd0d  readdata = 3b23f176
memtoregm = 1   resultw = 3b23f176
regwritem = 1   regwritew = 1
writeregm = 1e  writeregw = 1e

Test Case           4
aluoutw = 76d457ed  readdata = 462df78c
memtoregm = 0   resultw = 76d457ed
regwritem = 1   regwritew = 1
writeregm = 1c  writeregw = 1c

$finish called at time : 6 ns : File "C:/Users/fallo/Desktop/CECS 440/Lab 7/Lab_7/Lab_7.srcs/sim_1/new/t_WB_Stage.v" Line 33
```

Appendix (3 out 3)

Code

Mux.v

```
//Name: Eric Santana
//ID: 015107467
//Lab: 7
//Class: CECS 440

`timescale 1ns / 1ps
`include "mux.v"

module WB_Stage(
    input        regwritem, memtoregw,
    input  [31:0] aluoutw, readdata,
    input  [4:0]  writeregm,
    output        regwritew,
    output [4:0]  writeregw,
    output [31:0] resultw
);

    // Pass-through control signals
    assign regwritew = regwritem;
    assign writeregw = writeregm;

    // Mux selects between ALU result and Read Data
    mux #(32) resmux (
        .d0(aluoutw),
        .d1(readdata),
        .s(memtoregw),
        .y(resultw)
    );

endmodule
```

WB_Stage.v

```
//Name: Eric Santana
//ID: 015107467
//Lab: 7
//Class: CECS 440

`timescale 1ns / 1ps
`include "mux.v"

module WB_Stage(
    input         regwritem, memtoregw,
    input  [31:0] aluoutw, readdata,
    input  [4:0]  writeregm,
    output        regwritew,
    output [4:0]  writeregw,
    output [31:0] resultw
);

    // Pass-through control signals
    assign regwritew = regwritem;
    assign writeregw = writeregm;

    // Mux selects between ALU result and Read Data
    mux #(32) resmux (
        .d0(aluoutw),
        .d1(readdata),
        .s(memtoregw),
        .y(resultw)
    );

endmodule
```

TestBench.v

```
//Name: Eric Santana
//ID: 015107467
//Lab: 7
//Class: CECS 440

`timescale 1ns/1ps
module t_WB_Stage();
    reg        regwritem, memtoregm;
    reg [31:0] aluoutw, readdata;
    reg [4:0]  writeregm;
    wire       regwritew;
    wire [4:0] writeregw;
    wire [31:0] resultw;
    integer i;

    // Instantiate the WB_Stage module
    WB_Stage dut (
        .regwritem(regwritem),
        .memtoregw(memtoregm),
        .aluoutw(aluoutw),
        .readdata(readdata),
        .writeregm(writeregm),
        .regwritew(regwritew),
        .writeregw(writeregw),
        .resultw(resultw)
    );

    // Generate random test cases
    initial begin
        for (i = 0; i < 5; i = i + 1) begin
            aluoutw   = $random;
            readdata  = $random;
            {writeregm, regwritem} = $random;
            memtoregm = i;
            #1 testcase();
        end
        #1 $finish;
    end

    // Task: verify outputs
    task testcase();
    begin
        $display("Test Case %d", i);
        $display("aluoutw = %h\treaddata = %h", aluoutw, readdata);
        $display("memtoregm = %h\tresultw = %h", memtoregm, resultw);

        if ((!memtoregm && resultw != aluoutw) || (memtoregm && resultw != readdata)) begin
            $display("TEST FAILED: resmux has malfunctioned");
            $finish;
        end

        $display("regwritem = %b\tregwritew = %b", regwritem, regwritew);
        $display("writeregm = %h\twriteregw = %h", writeregm, writeregw);

        if ((regwritem != regwritew) || (writeregm != writeregw)) begin
            $display("TEST FAILED: signal passing malfunctioned");
            $finish;
        end

        $display("");
    end
    endtask
endmodule
```

Use of AI Tools:

ChatGPT Edu (GPT-5) was used to proofread the lab report for clarity and formatting.